data-pipelining-with-coinbase

September 15, 2026

0.1 Post Updates

Update 8/8/2026: Reworked `coinbase_pull_data` so that a new record is built from 2010 forward and an existing record resumes from its last stored candle, rather than the initial pull starting at a fixed year and updates re-fetching a caller-supplied range. Revised the walkthrough of that function to match.

Update 8/6/2026: Clarified how the initial pull differs from an incremental update, documented the asset filtering performed by `coinbase_pull_data`, reordered the function list to build from the lowest-level function upward, and replaced the historical candles example with an hourly range that returns more than a single row.

0.2 Introduction

This post covers how I collect and store minute, hour, and daily crypto asset data from Coinbase. Since I first published it I have expanded the walkthrough to cover how the initial pull differs from an incremental update, and the filtering that gets applied to the asset list along the way.

The work is split across four functions that build on one another. `coinbase_fetch_historical_candles` wraps a single request to the Coinbase Exchange candles endpoint. `coinbase_fetch_full_history` pages that endpoint to cover an arbitrary date range. `coinbase_fetch_available_products` lists the tradeable pairs. And `coinbase_pull_data` combines all three to maintain a directory of pickle files on disk, resuming an existing data file from its last stored candle, or building a new record from scratch where none exists.

The first three functions are executed below. `coinbase_pull_data` is shown but not run, since it writes to the filesystem and can run for a long time against a full asset list.

0.3 Python Imports

[1]:
```
# Standard Library
import datetime
import os
import sys
import warnings

from datetime import datetime
from pathlib import Path

# Suppress warnings
```

```
warnings.filterwarnings("ignore")
```

[2]:
```
# Add the source subdirectory to the system path to allow import config from␣
 ↪settings.py
current_directory = Path(os.getcwd())
website_base_directory = current_directory.parent.parent.parent
src_directory = website_base_directory / "src"
sys.path.append(str(src_directory)) if str(src_directory) not in sys.path else␣
 ↪None

# Import settings.py
from settings import config

# Add configured directories from config to path
SOURCE_DIR = config("SOURCE_DIR")
sys.path.append(str(Path(SOURCE_DIR))) if str(Path(SOURCE_DIR)) not in sys.path␣
 ↪else None

# Add other configured directories
BASE_DIR = config("BASE_DIR")
CONTENT_DIR = config("CONTENT_DIR")
POSTS_DIR = config("POSTS_DIR")
PAGES_DIR = config("PAGES_DIR")
PUBLIC_DIR = config("PUBLIC_DIR")
SOURCE_DIR = config("SOURCE_DIR")
DATA_DIR = config("DATA_DIR")
DATA_MANUAL_DIR = config("DATA_MANUAL_DIR")
```

0.4 Python Functions

Here are the functions needed for this project, listed in the order they build on one another:

- coinbase_fetch_available_products: Fetch available products from Coinbase Exchange API.
- coinbase_fetch_historical_candles: Fetch historical candle data for a given product from Coinbase Exchange API.
- coinbase_fetch_full_history: Fetch full historical data for a given product from Coinbase Exchange API.
- coinbase_pull_data: Update existing record or pull full historical data for a given product from Coinbase Exchange API.

[3]:
```
from coinbase_fetch_available_products import coinbase_fetch_available_products
from coinbase_fetch_historical_candles import coinbase_fetch_historical_candles
from coinbase_fetch_full_history import coinbase_fetch_full_history
from coinbase_pull_data import coinbase_pull_data
```

0.5 Function Usage

0.5.1 Coinbase Fetch Available Products

This function pulls the list of available assets based on the inputs for base and quote currency. In the example below the `quote_currency` is provided as "USD", so the function returns every asset priced against USD:

[4]:
```
df = coinbase_fetch_available_products(
    base_currency=None,
    quote_currency="USD",
    status="online",
)

display(df[["id", "base_currency", "quote_currency", "status"]])
```

```
            id base_currency quote_currency  status
655     00-USD            00            USD  online
155  1INCH-USD         1INCH            USD  online
497     2Z-USD            2Z            USD  online
815     A8-USD            A8            USD  online
563   AAVE-USD          AAVE            USD  online
..         …            …             …     …
608    ZKC-USD           ZKC            USD  online
835    ZKP-USD           ZKP            USD  online
778   ZORA-USD          ZORA            USD  online
282    ZRO-USD           ZRO            USD  online
20     ZRX-USD           ZRX            USD  online

[402 rows x 4 columns]
```

The API returns eighteen columns per product, but the four shown above are the ones that matter downstream, and are the same four that `coinbase_pull_data` prints when it builds its working list.

0.5.2 Coinbase Fetch Historical Candles

This is the lowest-level of the four functions, and maps to a single request against the Coinbase Exchange candles endpoint. It returns the date/time, open, high, low, close, and volume for each period in the requested range. The example below pulls hourly candles from the start of 2025:

[5]:
```
df = coinbase_fetch_historical_candles(
    product_id="BTC-USD",
    start=datetime(2025, 1, 1),
    end=datetime(2025, 1, 1, 6),
    granularity=3_600,
)

display(df)
```

	time	low	high	open	close	volume
0	2025-01-01 00:00:00	93281.63	94321.25	93347.59	94209.02	337.581664
1	2025-01-01 01:00:00	93393.06	94224.30	94207.31	93423.47	198.579180
2	2025-01-01 02:00:00	93400.00	93894.04	93423.48	93882.09	154.768788
3	2025-01-01 03:00:00	93524.35	93885.00	93885.00	93635.54	127.391075
4	2025-01-01 04:00:00	93305.54	93637.33	93635.54	93358.58	257.394980
5	2025-01-01 05:00:00	93323.88	93708.79	93358.33	93572.80	146.990291
6	2025-01-01 06:00:00	93354.99	93575.73	93575.72	93551.21	138.098835

Two details are worth noting here. The endpoint returns at most 300 candles per request no matter how wide a range is asked for, which is the constraint that the next function exists to work around. And the response arrives in reverse chronological order, so the function reverses it before building the dataframe. Rate limiting (HTTP 429) is retried up to five times with an exponential backoff.

0.5.3 Coinbase Fetch Full History

This function pulls an arbitrary date range for a specified asset by paging the function above. Because a single request is capped at 300 candles, it walks the range in windows of `granularity * 300` seconds, advancing from the timestamp of the last candle it received and pausing 0.2 seconds between requests to stay within the rate limits. The individual responses are concatenated into one dataframe, so a range of years is handled exactly the same way as the single month below:

[6]:
```
df = coinbase_fetch_full_history(
    product_id="BTC-USD",
    start=datetime(2025, 1, 1),
    end=datetime(2025, 1, 31),
    granularity=86_400,
)

display(df)
```

	time	low	high	open	close	volume
0	2025-01-01	92743.63	94960.91	93347.59	94383.59	6871.738482
1	2025-01-02	94177.00	97776.99	94383.59	96903.19	10912.473840
2	2025-01-03	96016.63	98969.92	96905.48	98136.51	9021.885382
3	2025-01-04	97516.65	98761.02	98139.85	98209.85	2742.089606
4	2025-01-05	97250.00	98814.00	98209.85	98345.33	2377.921759
5	2025-01-06	97900.00	102500.00	98347.65	102279.41	15173.556068
6	2025-01-07	96105.11	102735.99	102279.41	96941.98	16587.286922
7	2025-01-08	92500.00	97254.35	96941.98	95036.63	14182.297395
8	2025-01-09	91187.00	95363.26	95033.18	92547.44	9712.378532
9	2025-01-10	92209.25	95862.92	92547.44	94701.18	12634.034078
10	2025-01-11	93804.05	94983.65	94701.48	94565.02	2638.699568
11	2025-01-12	93670.30	95383.84	94569.91	94509.62	2025.816130
12	2025-01-13	89028.64	95900.00	94507.24	94506.45	13094.863595
13	2025-01-14	94311.36	97353.29	94507.35	96534.96	11210.742267
14	2025-01-15	96400.00	100716.45	96534.97	100510.23	13610.747294
15	2025-01-16	97277.58	100880.00	100504.27	99981.78	12312.373669
16	2025-01-17	99937.81	105970.00	99981.46	104107.00	20518.309493

```
17 2025-01-18  102233.45  104933.15  104107.00  104435.00   7835.299918
18 2025-01-19   99518.00  106314.44  104435.01  101211.13  13312.636856
19 2025-01-20   99416.27  109358.01  101217.78  102145.43  32342.183113
20 2025-01-21  100051.00  107291.10  102145.42  106159.26  19411.234890
21 2025-01-22  103100.00  106431.34  106159.27  103667.11  10730.018962
22 2025-01-23  101200.01  106870.87  103659.60  103926.36  25064.864999
23 2025-01-24  102751.92  107200.00  103926.36  104850.27  12921.993614
24 2025-01-25  104104.00  105294.00  104866.13  104733.56   3404.853083
25 2025-01-26  102452.24  105478.80  104729.92  102563.00   4575.366115
26 2025-01-27   97715.03  103228.46  102565.28  102062.42  23647.141119
27 2025-01-28  100213.80  103770.85  102063.92  101290.00   9488.534295
28 2025-01-29  101275.60  104829.64  101290.01  103747.25  11403.202789
29 2025-01-30  103289.74  106484.77  103747.25  104742.64  13061.348812
30 2025-01-31  101506.00  106090.00  104742.63  102411.26  13313.681045
```

0.5.4 Coinbase Pull Data

This function combines the three above to perform the following:

1. Resolve the list of products to update from the `base_currency` and `quote_currency` arguments.
2. Attempt to read an existing pickle data file for each product.
3. If a data file exists, resume from its last stored candle and pull forward to the present.
4. Otherwise, pull the available historical data for that asset and create a new record.
5. Store pickle and/or excel files of the data in the specified directories.
6. Prune the resulting product list down to the assets worth carrying forward.

Through the `base_directory`, `source`, and `asset_class` variables the function knows where in the local filesystem to look for an existing pickle file and where to store the resulting updated pickle and/or excel files:

```
df = coinbase_pull_data(
    base_directory=DATA_DIR,
    source="Coinbase",
    asset_class="Cryptocurrencies",
    excel_export=False,
    pickle_export=True,
    output_confirmation=True,
    base_currency="BTC",
    quote_currency="USD",
    granularity=60, # 60=minute, 3600=hourly, 86400=daily
    status='online', # default status is 'online'
)
```

Note that no dates are passed. `start_date` defaults to 1/1/2010 and only applies when there is no existing record, and `end_date` defaults to one day ago to account for the lag in data availability, so a single call per product and granularity is enough to either create a record or bring one current.

By passing `None` as the `base_currency` and/or the `quote_currency`, the function uses `coinbase_fetch_available_products` to build the product list, so a single call can pull every

USD-quoted pair on Coinbase.

On an update, the existing record is read back and the fetch resumes from the last date it contains. That final candle is deliberately re-fetched, since the period may still have been in progress when it was written; where a date appears in both the stored and the newly fetched data, the new row wins, so a partial candle is replaced by its completed version rather than left stale. The same range can therefore be pulled repeatedly without duplicating or corrupting anything.

The initial pull works differently. With no pickle file to resume from, there is no last date to start at, so the function falls back to `start_date` and works forward a year at a time until the API returns something. Coinbase has nothing for any of these pairs in 2010, so the first several attempts come back empty and are skipped; for BTC-USD the data begins on 7/20/2015, which is where the record actually starts.

The function also prunes the product list it returns. Assets with no data available, assets whose closing price never exceeds the price threshold, stablecoins (USDT, USDC, PAX, DAI, PYUSD, GUSD), and wrapped coins (WAXL, WBTC) are all removed. What remains is the set of assets actually useful for analysis, which matters when the starting product list runs to several hundred entries.

The example above pulls the data for BTC-USD, and stores it in the following system directory:

DATA_DIR/Coinbase/Cryptocurrencies/Minute

And here is the filesystem tree output for the `Coinbase` directory:

```
$ tree Coinbase/
Coinbase/
   Cryptocurrencies
       Daily
           BTC-USD.pkl
           BTC-USD.xlsx
           ETH-USD.pkl
           ETH-USD.xlsx
           SOL-USD.pkl
           SOL-USD.xlsx
           XRP-USD.pkl
           XRP-USD.xlsx
       Hourly
           BTC-USD.pkl
           BTC-USD.xlsx
           ETH-USD.pkl
           ETH-USD.xlsx
           SOL-USD.pkl
           SOL-USD.xlsx
           XRP-USD.pkl
           XRP-USD.xlsx
       Minute
            BTC-USD.pkl
            ETH-USD.pkl
            SOL-USD.pkl
```

```
            XRP-USD.pkl

5 directories, 20 files
```

Note that the `Minute` directory holds only pickle files. Minute data runs to hundreds of thousands of rows per asset, which makes the excel export slow and unwieldy, so `excel_export` is left as `False` at that granularity.

0.6 References

1. Coinbase
2. Coinbase Exchange API, products endpoint: `https://api.exchange.coinbase.com/products`
3. Coinbase Exchange API, candles endpoint: `https://api.exchange.coinbase.com/products/{product_id`
4. Reusable & Extensible Python Functions For Financial Data Analysis